

Semi-Humanoid Robotics Software: A Comprehensive Literature Review

Multi-rate autonomy, real-time control, embodied AI and sim-to-real systems

2022–2026 literature synthesis

Abstract

Semi-humanoid mobile manipulators require software that connects semantic perception and language-conditioned action to contact-safe, deterministic physical execution. This review consolidates recent literature on Vision-Language-Action (VLA) models, perception–cognition–operation architectures, ROS 2 autonomy, Behavior Trees, whole-body planning, `ros2_control`, Zenoh middleware, micro-ROS, variable-impedance control and OpenUSD digital twins. The literature supports a multi-rate architecture: cognitive inference at approximately 5–10Hz; mission, navigation and manipulation services at 30–100Hz; robot integration services at 100–500Hz; and native real-time control and safety at 1kHz. The principal research gap is not a missing algorithm in any one layer, but insufficiently specified contracts between layers: trajectory interpolation from VLA outputs, continuous base–arm coordination, jitter-isolated MCU control, containerised cloud-to-edge delivery, and tactile-force safety vetoes. A research agenda is presented around these interfaces, their benchmarks and reproducible evaluation.

Keywords: mobile manipulation; ROS 2; VLA; Behavior Trees; whole-body control; `ros2_control`; Zenoh; micro-ROS; impedance control; digital twin.

1 Introduction

Semi-humanoid robotics combines mobile autonomy with upper-body dexterity. Its software problem is inherently heterogeneous: visual and language models provide useful but non-deterministic, low-rate action proposals, whereas motor commutation, contact regulation and safety monitoring require bounded timing and explicit failure responses. A viable system therefore cannot be an end-to-end policy alone; it must be a coordinated hierarchy of perception, task control, planning, middleware, firmware and safety.[1, 11, 7]

This review addresses five questions: Which software functions are mature enough to form a platform baseline? How should rates and interfaces be partitioned? What mechanisms support mobile bimanual planning and recovery? How should real-time and safety functions be isolated? Which sim-to-real and deployment practices permit reproducible field operation?

2 Methodology

2.1 Review scope and evidence classes

The review covers literature published or indexed from 2022 to 2026, plus the 2021 micro-ROS real-time executor reference retained because of its direct relevance to embedded scheduling. Evidence is organised into two complementary classes: emerging preprints that describe current VLA, world-model, tactile-skin, whole-body and simulator approaches; and scholarly/technical literature that frames ROS 2 autonomy, Behavior Trees, DMPs, reconfiguration and human–robot digital twins. Reported citation counts, model configurations, data volumes and acceleration figures are reproduced as stated by the reviewed works and should not be interpreted as independently reproduced benchmarks.

2.2 Synthesis protocol

Each work was assigned to one or more of five layers: cognition and perception; task orchestration and planning; middleware and reconfiguration; firmware and safety; and simulation/digital twins. Cross-layer requirements were then extracted: timing, command semantics, state ownership, recovery behaviour, observability and deployment. This organisation avoids presenting a technology list as an architecture.

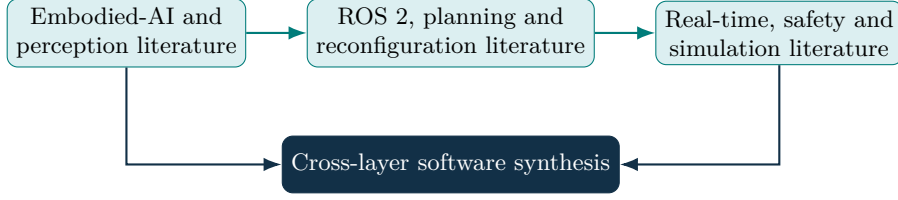


Figure 1: Review methodology: layers are analysed separately and then reconciled through their execution and safety interfaces.

3 Software taxonomy and execution model

3.1 Multi-rate stack

The literature consistently motivates a decoupled multi-rate pipeline. Cognitive services interpret intent and scene state; task services select recoverable skills; planning and integration services generate state-consistent trajectories; and local control executes only bounded commands. Table 1 formalises the taxonomy.

Table 1: Multi-rate software taxonomy for semi-humanoid mobile manipulators.

Layer	Indicative rate	Compute domain	Representative functions
Cognition and perception	5–10Hz	Edge GPU	OpenVLA/OpenVLA-OFT, panorama-aware VLA, JEPA-WAM, YOLOv11, SAM, speech-to-text, semantic 3D grounding
Task and planning	30–100Hz	Host CPU/GPU	BehaviorTree.CPP, Nav2, MoveIt 2, DMPs, whole-body SDF/MPC, action graphs
Integration and communications	100–500Hz	Linux host / middleware	<code>ros2_control</code> , joint-state/control interfaces, DDS or <code>rmw_zenoh</code> , iG-LIO, WMS/IoT and lifecycle management
Real-time and safety	1kHz	MCU/RTOS or PREEMPT_RT	FOC, joint/impedance loops, deadline monitor, tactile-force veto, E-stop and fieldbus services
Simulation and operations	asynchronous	Workstation/cloud/edge	OpenUSD assets, Isaac Lab, event-camera simulation, training, containers, OTA and telemetry

Evidence: Rates are architecture targets derived from the reviewed literature; deployment must establish measured WCET, jitter and end-to-end latency.

4 Cognition, perception and action representation

4.1 Vision-language-action models and world models

OpenVLA maps image and language inputs to 7-DoF action deltas and is reported to combine a 7B Llama 2 backbone with DINOv2 and SigLIP, pretrained on 970k Open X-Embodiment episodes. OpenVLA-OFT is reported to deliver $26\times$ inference acceleration for edge fine-tuning.[1, 2] Panorama-aware VLA extends this premise to dynamic mobile viewpoints; JEPA-WAM predicts stage-level visual outcomes for multi-step manipulation; and semantic 3D Gaussian Splatting grounds open-vocabulary language targets in 3D.[3, 4, 5]

These methods are valuable action-proposal generators, not direct motor controllers. A formally useful interface represents an AI output as $(g, \Delta\mathbf{x}, \Sigma, c)$: symbolic goal g , candidate Cartesian delta $\Delta\mathbf{x}$, uncertainty

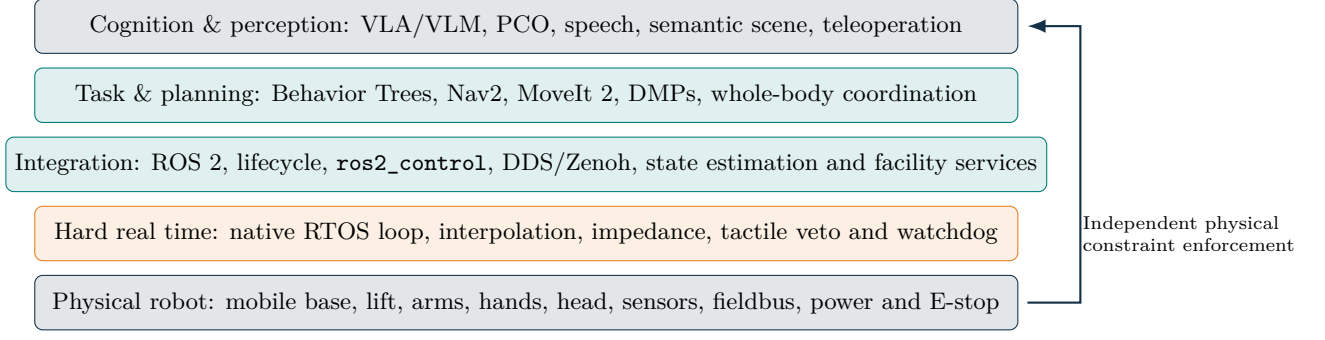


Figure 2: Reference architecture. The orange layer is the non-bypassable boundary between high-level software and physical actuation.

Σ , and confidence c . The autonomy system accepts only a projection into a verified feasible set:

$$\mathbf{x}_{\text{ref}} = \Pi_{\mathcal{F}(\mathbf{q}, \mathcal{E}, \mathcal{S})}(\mathbf{x} + \Delta \mathbf{x}), \quad (1)$$

where $\mathcal{F}$ includes kinematic, collision, payload, speed-zone, environment and safety-state constraints.

4.2 Perception–cognition–operation and interaction

The PCO framework provides a useful decomposition between perceptual estimation, semantic reasoning and motion operation.[6] In a semi-humanoid system, this accommodates RGB-D/LiDAR odometry, object detection/segmentation, language parsing, head orientation and human-facing feedback without collapsing all decisions into one learned policy. The perception layer should emit timestamped state estimates with frame IDs, confidence and freshness; downstream skills can reject stale or ambiguous evidence.

5 Task autonomy, navigation and whole-body planning

5.1 Behavior Trees, DMPs and skill graphs

Behavior Trees have a well-established role in modular reactivity and fault recovery relative to monolithic state machines.[7] DMPs provide compact trajectory representations that can be adapted under perturbation.[8] RoboReact adds a reported route from egocentric demonstrations to executable ROS 2 action graphs.[9] Together, these methods support a layered task representation: Behavior Trees select and recover skills; DMPs/trajectory generators parameterise motion; and planners validate execution against current geometry.

5.2 Nav2, MoveIt 2 and base–arm coupling

Nav2 provides SLAM/localisation, route generation, costmaps and local planning such as DWB and MPPI. MoveIt 2 provides collision-aware inverse kinematics and trajectory planning for dual 6–7 DoF arms. The mobile base can be represented as a virtual (x, y, θ) joint in an SRDF/whole-body model. The central challenge is coordination: independent Nav2 and MoveIt execution can yield stop-then-move behaviour. Kinematically coupled SDF/MPC approaches aim to co-optimize base velocity and arm limits for payload-aware mobile manipulation.[10, 11]

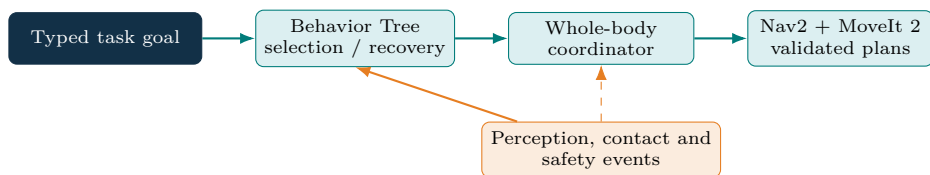


Figure 3: Recoverable skill execution. The task layer receives asynchronous events and may re-plan, degrade or request assistance.

6 Middleware, reconfiguration and robot integration

ROS 2 supplies modular transport, discovery and lifecycle semantics for the software ecosystem. The `ros2_control` abstraction standardises hardware interfaces, joint-state broadcasting and effort/position command distribution across heterogeneous VESC, ODrive and servo-drive actuators. Zenoh/`rmw_zenoh` is presented as an alternative for local zero-copy shared-memory communication and efficient wireless publish/subscribe, whereas DDS remains an established ROS 2 default. Dynamic reconfiguration and lifecycle management address controlled node replacement and recovery without whole-system reboot.[12]

QoS must distinguish data freshness from delivery certainty: RGB-D/point-cloud paths favour bounded best-effort delivery; joint state, action result, configuration and safety state require reliable, deadline-aware contracts. Neither DDS nor Zenoh replaces the hardware safety path. iG-LIO and autonomous pallet picking provide examples of ROS 2 localisation and object-alignment integration, while WMS/IoT telemetry and QML/operator interfaces expose the facility-facing side of the stack.[13, 14]

7 Real-time firmware, impedance and safety

7.1 Native control versus middleware callbacks

Hard real-time loops should run in native FreeRTOS/Zephyr or bare-metal MCU tasks, decoupled from micro-ROS callbacks. The micro-ROS budget-based executor is relevant for command and telemetry scheduling, but XRCE-DDS serialization and callback scheduling must not determine a 1kHz motor deadline.[15] A priority structure places E-stop/deadline/overcurrent handling above servo control; servo and contact regulation above fieldbus processing; and fieldbus processing above telemetry and parameter updates.

7.2 Impedance, tactile sensing and force veto

VIDP is reported as a diffusion-policy approach for compliant, variable-impedance manipulation from diverse demonstrations.[16] Conformal EIT touch skin provides a candidate geometry-scalable whole-body collision-sensing modality.[17] At the actuator boundary, the local controller uses a bounded impedance law,

$$\mathbf{F}_{\text{cmd}} = \mathbf{K}(\mathbf{x}_d - \mathbf{x}) + \mathbf{D}(\dot{\mathbf{x}}_d - \dot{\mathbf{x}}), \quad \tau_{\text{cmd}} = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + \tau_{\text{ff}}, \quad (2)$$

and independently saturates force, torque, velocity, acceleration and power. A tactile/force veto must transition the system into an explicit safe hold, compliant retreat or latched fault state; the cited 25N value is a proposed configuration point, not a universal safety threshold.

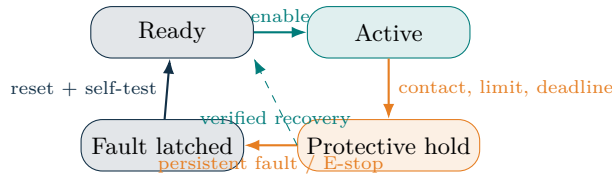


Figure 4: Safety state model for software-controlled execution. Hardware E-stop and power isolation remain independent protective measures.

8 Digital twins, simulation and deployment

OpenUSD provides a scene and asset representation for Isaac Sim/Isaac Lab, procedural environment generation and sensor simulation. EsaacSim extends Isaac Sim with event-camera and dynamic sensor models for visual-servoing studies.[18, 19] Digital-twin literature for human–robot collaboration motivates real-time synchronisation, collision-aware evaluation and operator-aware simulation.[20]

The software lifecycle should promote a capability through model-in-the-loop, SIL, vHIL, supervised hardware and bounded field deployment. Domain randomisation should include lighting, textures, sensor noise/latency, friction, payloads, wheel slip and contact conditions. Containerised deployment closes the remaining gap: build immutable images for VLA/perception, Nav2, MoveIt 2 and `ros2_control`, then promote signed releases through simulation, bench, canary and fleet stages with telemetry and rollback.

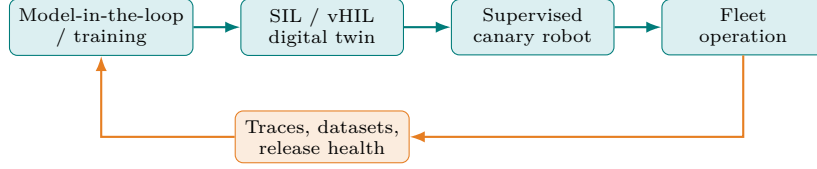


Figure 5: Sim-to-real and release lifecycle. Observability and rollback gates transform deployment into a controlled experiment.

9 Open problems and research agenda

Table 2: Unified software research agenda.

Research gap		Failure mode	Required research artefact
VLA-to-control disparity	rate	Raw 5–10Hz deltas yield discontinuous or unsafe execution	Trajectory interpolator plus variable-impedance filter; latency, smoothness and contact-safety benchmark
Base-arm coupling	kinematic	Nav2 and MoveIt 2 create stop-then-move or dynamically awkward behaviour	Open whole-body coordinator; payload, clearance and throughput comparison against sequential planning
Bimanual BT recovery		Single-arm/navigation trees do not manage slip, blocked transport or handover failure	Reusable recovery-node library and benchmark with recovery success/time-to-recovery
Micro-ROS timing jitter		Callback-driven control violates hard timing budgets	Native RTOS control task with measured WCET/jitter; micro-ROS used only as asynchronous bridge
Cloud-to-edge reproducibility		Simulator/host dependency drift causes deployment breakages	Open container manifests, hardware configuration schema, signed OTA and rollback evaluation
Vision-only blindness	VLA contact	Occlusion/unknown contact causes inappropriate action continuation	<code>ros2_control</code> -integrated tactile/F/T/EIT veto with fault-injection evidence

10 Conclusion

The literature establishes a robust vocabulary for semi-humanoid software: VLA and world models for semantic proposals; PCO for functional decomposition; Behavior Trees and DMPs for recoverable task execution; Nav2, MoveIt 2 and whole-body planning for mobile manipulation; ROS 2, `ros2_control` and Zenoh/DDS for integration; native MCU firmware for hard timing; and OpenUSD digital twins for validation. The outstanding research contribution is the verified connection of these layers. Future systems should demonstrate, rather than assume, that high-level intelligence is constrained by time-stamped plans, that middleware is isolated from servo deadlines, that tactile contact can veto vision-derived actions, and that releases transfer from simulation to fleets with traceable evidence.

References

- [1] M. J. Kim et al., “OpenVLA: An Open-Source Vision-Language-Action Model,” arXiv:2406.09246, 2024. <https://arxiv.org/abs/2406.09246>.
- [2] M. J. Kim et al., “Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success,” arXiv:2502.19645, 2025. <https://arxiv.org/abs/2502.19645>.

- [3] D. Yang, H. Chen, and X. Chen, “Learning Panorama-Aware VLA for Mobile Manipulation with Whole-Body Teleoperation,” arXiv:2608.02257, 2026. <https://arxiv.org/abs/2608.02257>.
- [4] X. Liu, Y. Yang, and X. Wang, “JEPA-WAM,” arXiv:2608.10780, 2026. <https://arxiv.org/abs/2608.10780>.
- [5] H. Ou, D. Song, and Y. Wang, “Embodied Multimodal Grounding via Semantic 3D Gaussian Splatting,” arXiv:2608.10756, 2026. <https://arxiv.org/abs/2608.10756>.
- [6] M. V. L. Carvalho, L. R. Yoshioka, and J. F. Justo, “The Perception–Cognition–Operation Approach,” 2026. DOI: <https://doi.org/10.20944/preprints202603.0528.v1>.
- [7] M. Iovino et al., “A Survey of Behavior Trees in Robotics and AI,” *Robotics and Autonomous Systems*, 2022. DOI: <https://doi.org/10.1016/j.robot.2022.104096>.
- [8] M. Saveriano, F. J. Abu-Dakka, and A. Kramberger, “Dynamic Movement Primitives in Robotics,” *IJRR*, 2023. DOI: <https://doi.org/10.1177/02783649231201196>.
- [9] T. He et al., “RoboReact,” arXiv:2608.03387, 2026. <https://arxiv.org/abs/2608.03387>.
- [10] Y. Li, L. Yin, and T. Zhang, “Kinematically-Coupled SVSDF Whole-Body Planning,” arXiv:2608.07005, 2026. <https://arxiv.org/abs/2608.07005>.
- [11] A. Bonci, F. Gaudeni, and M. Giannini, “ROS2-Based Frameworks for Increasing Robot Autonomy,” *Applied Sciences*, 2023. DOI: <https://doi.org/10.3390/app132312796>.
- [12] S. Peldszus, D. Brugali, and D. Strüder, “Software Reconfiguration in Robotics,” arXiv:2310.01039, 2023. <https://arxiv.org/abs/2310.01039>.
- [13] A. E. Carvalho, D. Portugal, and P. Peixoto, “A Numerically-Robust ROS 2 Port of iG-LIO,” arXiv:2607.09947, 2026. <https://arxiv.org/abs/2607.09947>.
- [14] T. Mökkönen, “Autonomous Pallet Picking Using ROS2,” Tampere University, 2024. <https://trepo.tuni.fi/handle/10024/154752>.
- [15] N. Stoufs et al., “Budget-Based Real-Time Executor for Micro-ROS,” arXiv:2105.05590, 2021. <https://arxiv.org/abs/2105.05590>.
- [16] H. Khalil, N. Fernandes, and T. M. Kwok, “VIDP: Variable Impedance Diffusion Policy,” arXiv:2608.06210, 2026. <https://arxiv.org/abs/2608.06210>.
- [17] H. Chen, C. Kohlbrenner, and J. Kubik, “Geometry-Scalable Whole-Body Touch for Humanoids,” arXiv:2608.02080, 2026. <https://arxiv.org/abs/2608.02080>.
- [18] M. Mittal et al., “Isaac Lab,” arXiv:2403.07257, 2024. <https://arxiv.org/abs/2403.07257>.
- [19] I. Bugueno-Cordova et al., “EsaacSim,” arXiv:2608.08522, 2026. <https://arxiv.org/abs/2608.08522>.
- [20] A. K. Ramasubramanian, R. Mathew, and M. Kelly, “Digital Twin for Human–Robot Collaboration in Manufacturing,” *Applied Sciences*, 2022. DOI: <https://doi.org/10.3390/app12104811>.